Are Long-Running Agents Making Progress? A Longitudinal Study of Artifact Progress, Repeated Mutation, and Continuity

Anonymous submission

Abstract

Long-running agents can modify a workspace for days across many session boundaries. When they stop, action counts, commits, and final artifacts do not reveal whether activity accumulated into durable and validated progress or repeatedly overwrote, abandoned, and rediscovered the same work. Existing trajectory studies characterize actions within benchmark attempts, while studies of long-lived agents focus on task outcomes or internal memory degradation. We instead make the persistent workspace the longitudinal unit of analysis. We introduce a source-linked projection that connects native Claude, Codex, and Gemini actions to stable artifact identities, lifecycle effects, event time, and session boundaries without introducing semantic labels or aligning the timeline to Git commits. We use it to study six longitudinal questions over six real software and autonomous-research projects—introduced-artifact persistence, later reuse, validation association, repeated mutation, workspace-activity migration, and cross-session continuity—plus a seventh question about tool-call workload and conservative optimization-opportunity bounds. Under the admitted local projection, across 551 native root sessions and 181,303 Tool actions, 89.3–97.1% of eligible mutations are later revisited, yet action volume only weakly orders the cross-project reuse relation ($\rho = 0.20$). Repeat-observed episodes account for 74.6–90.8% of mutation episodes, with 42.0–86.7% of episodes concentrated in the most-mutated 10% of identities. Source coverage is itself consequential: recognized successful validation appears in all six cases after projection repair, and named Skills show no supported repeatable separation in the only eligible comparison. In 320 stratum-specific public task/topic selections, path-local transitions and 2–3-intervening-call module returns recur, but persistent artifact lineage and cross-session re-grounding remain unobservable. Native overlap already consumes 86.0% of logical-parallel edges, while an actionable next-read prefetcher is only 21.75% precise. Finally, a source-direct experiment moves from 32/60 to 60/60 artifact-linked plus cross-session facts after repair on its corrected repair corpus; a preregistered 70-session held-out test instead obtains 54/58 and attempted-edge F1 0.9950. These are corpus-specific conformance results, not a general exact-fact capability claim.

Introduction

Long-running agents increasingly receive a broad goal, work in one repository for days, and return a workspace containing code, prose, tests, and experimental output.

Tool calls and commits show activity, but not whether that activity accumulated into durable, reused, and validation-associated progress. Final state omits reads, failed validations, transient artifacts, overwritten attempts, and the work of re-establishing context after a session boundary. Raw logs preserve these events but not the cross-session artifact lineage needed to connect them. We use "progress" strictly in the descriptive sense of this paper: whether observed activity consolidates into persistent, revisited, and validation-associated artifacts—not as a judgment of outcome quality, correctness, or developer productivity.

Trajectory studies already characterize action sequences, validation behavior, and destructive revision in benchmark attempts (Bouzenia and Pradel 2025; Mehtiyev and Assunção 2026; Kim et al. 2026). ProcGrep represents such traces as action procedures (Oderinwale 2026); persistent workspaces and multi-session reliability are also established settings (Zhou et al. 2026; Zhu et al. 2026). Context management can rapidly weaken earlier plans (Mehta and Datta 2026). The remaining gap is longitudinal and artifact-centered: a project persists while its sessions end, and its files acquire histories that attempt-level action summaries do not express.

We therefore keep three observable dimensions separate. The persistent workspace, rather than an isolated session or commit, is the longitudinal unit that makes these dimensions jointly observable. *Introduced-artifact persistence* asks whether a confirmed introduction survives in the final workspace; *reuse* asks whether later source-linked work returns to the same artifact lineage; and *validation association* asks whether an adapter-recognized successful check occurs before the artifact's next mutation or deletion. Their conjunction is descriptive shorthand, not a weighted quality score. We likewise treat repeated edits, unvisited artifacts, and validation gaps as process facts rather than intent, waste, or failure labels.

Our source-linked projection retains native Claude, Codex, and Gemini Tool actions, resolves qualified lifecycle effects, preserves explicit rename and session lineage, and orders events by Agent action time. Git contributes final tracked state but never defines the timeline. The same source facts support

the quantitative analyses and auxiliary AGENT NEBULA replay; visual coordinates never enter the measurements. Independent source-direct checkers test the projection rather than accepting its rows as truth.

Across 551 native root sessions and 181,303 Tool actions in six projects, eligible-mutation reuse is 89.3–97.1% but action volume only weakly orders the six-case reuse relation ($\rho = 0.20$). Repeat-observed mutations are common and concentrated; recognized successful validation covers all 6/6 cases after repair. The only eligible Skill comparison shows no repeatable fingerprint, while public traces reproduce path locality and 2–3-call module returns but not persistent lineage. Shell carries 68.6% of calls, while native overlap already consumes 86.0% of logical-parallel edges and actionable next-read prefetch is only 21.75% precise. Finally, source-direct conformance moves from 32/60 to 60/60 artifact-linked plus cross-session facts on the corrected repair corpus, but a preregistered 70-session held-out test obtains 54/58.

This paper contributes (1) a workspace-centered longitudinal characterization of artifact progress, continuity, and tool-call workload bounds (Section); (2) a deterministic, source-linked protocol with explicit measured, coverage-only, disagreement, and N/A states (Section); and (3) a conformance gate whose frozen 32/60 artifact-linked and cross-session result motivated root-cause repair to 60/60 on its corrected repair corpus and a separate 54/58 held-out test (Section). The supplement contains complete definitions, per-project results, all moved figures, and the extended validity analysis.

Workspace-Centered Measurement

The pipeline parses native sessions, admits complete repository-linked source universes, projects qualified effects onto stable artifact lineages, and derives project-level outcomes. Its requirements are source linkage, native action order, preserved failed calls and missingness, stable cross-session identity, and answers that an independent source reader can reproduce or mark undefined.

Source universe and order. A workspace W persists while native root sessions $S = (s_1, \ldots, s_m)$ issue Tool actions $A = (a_1, \ldots, a_n)$. Sessions partition model context but do not reset W. We admit a complete session when its project directory, worktree, or Git remote identifies a selected repository, and retain every Tool action in that session, including failed, no-path, and search actions. Native timestamp and stable source ID order the trace; global path-only matches remain a separate sensitivity source.

Action and effect projection. For each action we retain

$$a_i = (t_i, id_i, s_i, v_i, u_i, e_i, q_i), \quad E_i = \{(w, f, o, f')\}, \quad (1)$$

where vendor v_i, Tool/category u_i, adapter-derived command effect e_i, and status q_i accompany zero or more effects on worktree w and relative artifact f. Operation o is read, write, create, rename, or delete; f' exists only for an observed rename. Failed calls remain actions but emit no confirmed-success effect. Missing fields and unresolved paths remain coverage gaps.

Artifact lineage and outcomes. Identity is a hashed worktree and relative path. Tool-level workdir overrides session cwd; an explicit same-worktree rename preserves lineage and birth state; delete followed by create starts a new identity. Only a confirmed-success create is an eligible introduction. For every confirmed non-delete mutation, we locate later access and the next recognized successful test, check, or build before another same-artifact mutation or deletion. Supersession is a competing outcome; only observation end is right-censored. Final existence and tracked state are independently queried in the associated worktree. A recognized success establishes temporal association, not mutation-level coverage or correctness.

Observable progress without semantic gold. The projection supports three source-verifiable outcomes. Persistence uses final existence or tracked state only for artifacts observably introduced during the trace; writes to pre-existing files have unknown content durability without independent diffs or snapshots. Reuse requires a later access to the same lineage and retains event, time, and session distance. Validation association measures distance from a mutation to the next recognized successful validation before supersession. Directory arguments are weak scope evidence rather than fabricated reads of all descendants, and unknown status is neither success nor failure. Complete distance curves or declared sensitivity ranges replace a fixed attention window. A never-revisited file is an observed orphan, not automatically useless; repeated edits remain uninterpreted without task evidence.

Protocol and implementation. Each repository is one complete case. Cross-case summaries state project weighting and never treat actions as independent projects; blocked bootstrap or permutation tests use session or project blocks only when exchangeability is defensible. Artifact class is a deterministic path/extension mapping and top-level directories are the default module unit. The Rust implementation extends an existing repository projection, reuses native `agent-session` parsing, and exports source-linked JSON/CSV rows. Structured events, rather than AGENT NEBULA layout coordinates, drive every statistic. Independent checkers reparse native sources for the conformance, Skill, and public-trace analyses.

Capability protocol. The separate measurement question holds source membership and cutoff state fixed, stratifies facts as action-only, artifact-linked, cross-session, or final-state, and requires every answer to cite source evidence or abstain. Final State and Counts are lower-information controls, pinned ProcGrep is the strongest action-procedure baseline, and a bounded Raw reader is a same-source reconstruction condition. No method labels its own oracle. Deterministic conditions complete 480 rows; the bounded Raw matrix materializes the remaining 360 denominator rows, so the planned 840-row denominator is complete under the registered terminal-status rules.

Empirical Study

Research questions. **RQ1** asks how introduced and repeatedly modified artifacts persist, receive later reuse, concentrate work, become dormant, and return over Agent action time.

RQ2 asks how successful and failed validations interleave with mutation bursts, backlog, and subsequent workspace changes. **RQ3** asks how path-resolved activity is allocated among artifact types and how artifact/module hotspots migrate and receive returns. **RQ4** asks what re-grounding and prior-artifact/module continuity is observable between adjacent non-overlapping native-root components. **RQ5** asks whether workspace-action composition attributed to named Skills is repeatable across independent roots and what follows explicit instruction-file access. **RQ6** asks which compatible within-attempt relations recur in public coding and scientific-process traces and which longitudinal claims remain N/A. **RQ7** asks how tool-call composition, repetition, dependency, and waiting structure bound prefetch, concurrency, speculation, caching, and event-driven control in persistent workspaces.

Cases and inclusion. We fix six author-associated local projects before computing process rates. Four are public MIT-licensed repositories from one open-source community, and two are private repositories. At a 2026-07-26 snapshot, the community's 144 public repositories had about 9.9K aggregate GitHub repository stars. Each public case had two or three verified active maintainers in its observation window; each private case had one. Admitted session starts span 191.8 days (about 6.3 months) overall. The cases cover systems software and research, tutorial/content software, paper/experiment code, and skill/harness development. Each repository remains one case, and the double-anonymous framing otherwise treats them only as author-associated local projects. The main corpus includes complete repository-direct Claude, Codex, and Gemini sessions and retains calls without resolved file effects. Global path-only matches are a separate sensitivity source because they omit surrounding no-file calls.

The corpus comprises 551 native root sessions, deduplicated from 2,049 source transcript files, and 181,303 Tool actions; 551 roots and 176,288 actions resolve to retained worktrees. The repaired projection yields 5,746 artifact identities and 13,906 confirmed mutation rows. All six cases pass the longitudinal gate, but each estimand retains its own source-coverage denominator. Primary outputs are project-level effect sizes, distributions, cumulative-incidence curves, and explicit coverage. Model-derived motifs or summaries are exploratory unless they resolve to source IDs and deterministic metrics.

Admission gates. The longitudinal gate requires admitted native roots plus worktree-attributed actions, artifacts, and mutations for a project-level case. Cross-case estimands proceed only when at least four projects have a defined denominator. RQ4 additionally requires 20 eligible boundaries within each contributing project, while an exact-context RQ5 Skill stratum requires at least three distinct native-root sessions. A stopped gate yields within-case or coverage evidence, never an imputed zero.

RQ1—Persistence, reuse, and repeated mutation. We ask whether observed introductions persist, whether later work returns to mutated lineages, and whether activity volume orders those outcomes. All six cases contain eligible confirmed introductions after projection repair and therefore support introduced-artifact persistence: the project proportions are 973/1042, 28/239, 18/30, 8/18, 18/18, and 1/1. Later reuse appears for 89.29–97.11% of eligible mutations. Its descriptive cross-project association with worktree-attributed action volume is only Spearman $\rho = 0.2000$, so action count weakly orders even this widely covered relation in the selected cases (Figure 1). Repeat-observed episodes account for 74.6–90.8% of mutation episodes, and the most-mutated 10% of identities account for 42.0–86.7%. These distributions establish recurring, concentrated work, not convergence or thrashing. Under two declared dormancy thresholds, 8.3–48.3% (> 100 intervening Tool actions) and 0.0–40.0% (> 24 elapsed hours) of observed artifacts later revive; the supplement reports per-project counts and gap distributions.

RQ2—Validation response. We ask how recognized successes and failures interleave with mutation bursts and worktree-local mutation accumulation. Projection repair raises recognized-success coverage from three to all six projects, so the predeclared four-project gate now admits a cross-case description. Recognized success covers 6/6 projects, 5/6 form complete inter-success intervals, and 4/6 contain a recognized failure. Across seven eligible worktree lanes in those five projects, 29.3–86.1% of complete intervals contain no mutation, while lane maxima range from 1 to 817 mutations. The corresponding per-project validation-before-supersession proportions are 2450/6112, 1210/5604, 141/694, 47/282, 9/196, and 33/247. This measures recognized cadence, not redundant testing or validation coverage of each mutation. Mutation effects project to the affected worktree, whereas a validation stays on the command worktree; incomplete intervals at observation end are not silently counted as complete. Outcome-conditioned response has no consistent cross-case direction: the three higher-event-volume success/failure cases usually validate again before mutating after failure, whereas the sparse three-failure case mutates first. RQ2 therefore finds that many adjacent recognized successes contain no intervening mutation, yet rare intervals accumulate long mutation bursts.

RQ3—Workspace focus evolution. We ask how path-resolved access and mutation are allocated and how artifact and module focus changes in native call order. Path-resolved calls are analyzed in native order as same-artifact, same-module, or cross-module transitions, with access and mutation and `ok` and `observed` statuses kept distinct. All six cases now carry return evidence after the native-root session-join repair, and pooled unknown-create births fall from 790 to zero. Status sensitivity can be material: in one case, mutation allocation to paper/docs is 39.2% over all resolved statuses but 88.2% for `ok` only, so neither view is silently promoted to ground truth. In the path-compatible local anchors used by RQ6, cross-module movement is 2.1–20.2%, so 79.8–97.9% of adjacent transitions stay on the same artifact or within its top-level module. Qualified local module returns have medians of 2–4 strictly intervening calls. RQ3 therefore finds predominantly path-local movement with short observed returns. The supplement additionally reports action-window rank turnover and hot-set retention (rank-based "cooling"), neither of which measures elapsed duration, internal attention,

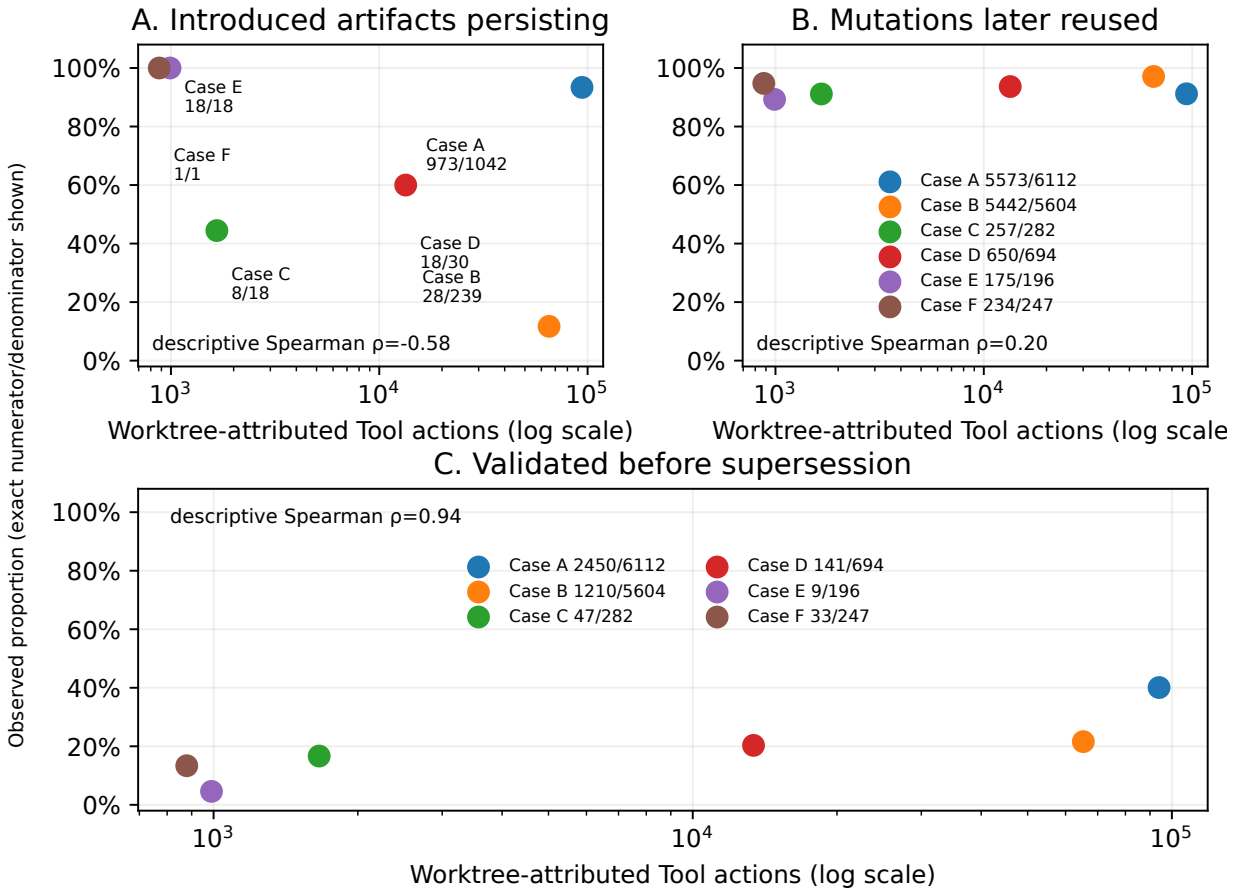

Figure 1: Activity volume versus the three separately reported RQ1 proportions. All dimensions cover six cases after repair; labels show exact numerators/denominators. Correlations are descriptive across six projects, not population estimates.

importance, productivity, or forgetting.

RQ4—Cross-session continuity. We ask what source-verifiable re-grounding and workspace continuity occurs between independent, temporally ordered session components. Streams sharing a native root remain one independent block, and transitive concurrency components prevent us from inventing order among overlapping sessions. Only adjacent non-overlapping components are compared for pre-mutation re-grounding and prior artifact/module overlap. The projection forms 121 components and 111 boundaries, but timing is defined only when a first mutation is observed and empty denominators remain undefined. After corrected root/subagent joining, only three of six projects reach 20 eligible boundaries; the preregistered four-project estimator gate therefore still stops. RQ4 is a data-limited within-case and coverage result, not evidence of reset cost, memory, comprehension, or forgetting.

RQ5—Skill and instruction footprints. We ask whether source-attributed named-Skill action composition is repeatable within exact contexts, without turning association into a harness effect. A separate checker reconciles all 2,063 included native transcript streams, including 7,304 Skill/instruction signal rows and 205,836 adjacent Tool boundaries. The cases contain 67 explicit Skill invocations and 1,675 source-attributed Skill Tool actions. Invocation and execution do not always share one transcript stream: only 476 attributed actions are contiguously preceded by a matching same-stream invocation, which is a coverage diagnostic rather than an inferred delegated episode. Five exact-context named-Skill strata qualify, but only one project supports a two-Skill comparison. There, median Jensen–Shannon distance is 0.116 for nine same-Skill pairs and 0.123 for ten different-Skill pairs; the one-sided exact root-block test gives $p = 0.750$ over 12 admissible assignments. Thus the only eligible comparison supports no repeatable Skill fingerprint. Accesses to `AGENTS.md`, `CLAUDE.md`, and `SKILL.md` remain separate focal events. Their immediate next Tool action before a prompt-index change is defined for 2,822 independently recomputed high-confidence direct or simple-shell accesses; project-normalized compositions are descriptive and do not prove injection, compliance, utility, or harm.

RQ6—External boundary. We ask whether compatible path-local relations recur outside the selected long-running cases and explicitly stop where public sources lack lineage.

We select 64 independent units within each of four Open-SWE-Traces harness/model strata and within the IdeaTrail topic stratum: 320 stratum-specific selections. Across strata, the 256 Open-SWE selections contain 255 unique task IDs, so independence is not claimed globally. An independent parser reconciles 31,249 Tool calls and 22,113 adjacent path-target transitions. Cross-module movement is 18.0–30.0% publicly versus 2.1–20.2% in compatible local anchors, and module-return medians are 2–3 intervening calls publicly versus 2–4 locally. Path locality and short returns recur, with a magnitude difference; persistent artifact lineage, cross-session regrounding, and exact Skill attribution are unavailable and remain N/A. Exploration-before-mutation is non-confirmatory because public harnesses shape action order, and recognized validation lacks a portable attempt/status pair. Path concentration and staged revision are analogies, not stable-artifact replication.

RQ7—Tool-call workload and optimization bounds. We ask what composition, repetition, dependency, and waiting structure the long-running tool workload exhibits, and what conservative opportunity bounds that structure permits. As observation bounds, shell accounts for 68.6% of calls, 46.7% of artifact-identity reads repeat within the same prompt and source stream, and 15.4% of shell calls rerun the exact command; none is a claim of avoidable work. As another observation bound, native overlap consumes 86.0% of logical-parallel edges. As opportunity bounds, a chronological actionable next-read prefetcher is only 21.75% precise, and replacing same-handle consecutive no-progress polls with one event-driven await could remove at most 1,456 calls. Across timing-complete episodes, 74.8% of observed span lies in between-tool gaps, which is an episode-level observation bound rather than model time or recoverable latency. RQ7 therefore finds a shell- and repetition-heavy workload whose explicit parallelism is mostly already consumed and whose remaining opportunities are conservative structural bounds, not realized performance or utility effects.

The supplement reports the full gate definitions, coverage tables, per-project denominators and curves, status sensitivities, checker procedures, the three detailed workload-estimator groups, and the nine empirical figures omitted here.

Measurement-Capability Conformance

We freeze 12 complete native files per project and derive 120 questions: 30 each about action-only (A), artifact-linked (B), cross-session (C), and final-state (D) facts. A standalone checker imports neither the projection nor `agent-session`; it reparses all 72 files, reconstructs 1,721 artifact edges, and verifies cutoff state.

Frozen failure. On 2026-07-23, the trajectory answered all 60 B+C questions but only 32 correctly, with per-project conditional accuracy ranging from 0.0 to 1.0. Final State answered all 30 D questions, while ProcGrep and Trajectory produced identical answers on all 30 A questions and matched the frozen source-direct grammar on 18; the official action adapter deliberately leaves some terminal reads unclassified.

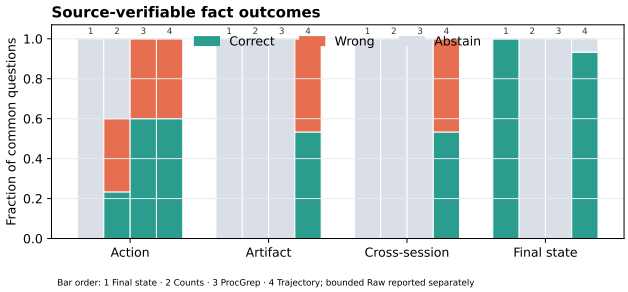

Figure 2: Correct, wrong, and abstaining outcomes on the corrected 120-question oracle for the deterministic conditions. Bounded Raw is evaluated separately under the registered fixed-reader protocol.

Taxonomy. Of the 28 disagreements, 14 arose from deliberately broader shell/scope evidence and 14 from genuine bugs (seven native-root joins, four failed-call effects, and three shell-path extractions); the audit also exposed an event-workdir bug. The independent oracle was corrected to v4, changing 24/120 expected answers with question-level source justification.

Repaired conformance. After fixing all four root-cause classes, the current revision scores B 30/30, C 30/30, and D 30/30 against v4 (Figure 2). Its A-family score is 12/30 because the trajectory deliberately preserves ProcGrep's action spine while v4 re-derives atoms under a source-direct grammar. The result is 60/60 B+C repair-corpus conformance, not a general exact-fact capability claim.

Root-disjoint held-out conformance. After that repair-corpus regression, a preregistered corpus instantiates 116 new questions over 70 root-disjoint native-session roots. B+C scores 54/58; attempted-edge precision/recall/F1 is 0.9906/0.9995/0.9950, and confirmed-edge precision/recall/F1 is 0.9903/0.9995/0.9949. The four question errors and 20 row-level edge differences localize to compound shell and wrapper path-admission boundaries. This valid negative held-out result is reported separately: the earlier 60/60 remains repair-corpus regression evidence, is not pooled with 54/58, and does not support general exact conformance. The supplement gives every held-out B+C item and the complete ledger summary.

Final State, Counts, and pinned ProcGrep remain lower-information controls. In the fixed-reader, fixed-corpus comparison, Raw obtains 191/360 (53.1%) overall exact coverage and 94/180 (52.2%) B+C coverage. It produces 260/360 (72.2%) scoreable rows, with 191/260 (73.5%) exact accuracy among scoreable rows; 5/18 cells trigger the preregistered 1 MiB tool-return limit. The comparison is mixed/inconclusive and supports no Trajectory superiority, necessity, speed, or cost claim. State Diff, Session Local, and OCPM Features remain N/A on this matrix.

Threats to Validity and Ethics

Native records may omit subprocess effects, external edits, and uninstrumented actions; missing fields are coverage gaps, not

no-effect observations. Final existence cannot prove content survival, and recognized success is association rather than coverage or correctness. Optional system-level evidence is a future coverage extension, not a second truth layer in this study. Existing-file writes retain unknown content durability unless native diffs, snapshots, or Git evidence support it independently.

The conformance audit found material disagreement rather than assuming the projection correct. The repair covers genuine path, identity, root-join, failed-effect, and workdir errors; we then recomputed RQ1–RQ4. The repaired 60/60 B+C result is a repair-corpus regression, while the separate root-disjoint held-out test scores 54/58 and exposes residual compound shell/wrapper path-admission boundaries. The two results are not combined, and the action-only grammar difference remains explicit. RQ5 and RQ6 are independently reconstructed, but Skill, model, task, phase, and project remain confounded. Repeated actions, unvisited artifacts, and long validation gaps are observable process facts, not automatic failure labels.

The six cases are selected from local author-associated projects. They offer deep source-native histories but cannot estimate population rates; public Open-SWE-Traces attempts are synthetic, and IdeaTrail is reverse-synthesized under one workflow. They replicate only compatible within-attempt relations and cannot validate natural cross-session persistence, re-grounding, or Skill effects. Native trajectories can contain private prompts, code, paths, secrets, and experimental data. Release requires repository and data permission, credential and personal-content removal, and aggregate or explicitly approved examples. Process measures must not become human-productivity scores: we do not rank people, equate activity with progress, or label repeated work as waste without independent task evidence. The intended use is analysis by an Agent or automated diagnostic process over its own authorized workspace.

Related Work

Software-agent trajectory studies measure actions, testing, and strategy (Bouzenia and Pradel 2025; Mehtiyev and Assunção 2026); TRAJEVAL identifies coherence collapse after useful intermediate code (Kim et al. 2026); and ProcGrep provides action procedures and fingerprints (Oderinwale 2026). AgingBench and plan-persistence work study long-session reliability and context (Zhu et al. 2026; Mehta and Datta 2026), while OR-Space supplies persistent multi-artifact tasks (Zhou et al. 2026). AgentDiagnose and AgentRx diagnose completed traces (Ou et al. 2025; Barke et al. 2026); TrajAudit retrieves trace evidence (Wang, Xie, and Huo 2026); and HarnessFix links failures to harness mechanisms (Chen et al. 2026). We do not claim the first action analysis, fingerprint, persistent workspace, or diagnostic labeler. Our contribution is source-linked artifact identity and cross-session lineage for measuring real project histories.

Conclusion

Persistent workspaces, not isolated sessions, expose whether long-running Agent activity survives, receives later reuse and recognized validation, concentrates in repeatedly changed artifacts, and continues across session boundaries. Six cases show high later reuse but weak ordering by action volume, alongside hard coverage limits for continuity and Skills. Public traces reproduce path locality and short module returns but not longitudinal lineage. The workload study finds that native overlap already consumes most explicit logical parallelism, leaving only conservative opportunity bounds. Finally, a frozen 32/60 B+C failure led to 60/60 on the corrected repair corpus, while a separate preregistered root-disjoint test obtains 54/58 and localizes residual mismatch to compound shell/wrapper paths. Neither result implies general exact conformance.

Data availability. Exported source-linked rows, the 120-question conformance benchmark with expected answers, and all analysis code are released in the anonymized repository (link on OpenReview). Raw native session records contain private prompts and filesystem paths and are not redistributable; every reported statistic is recomputable from the released rows without them.

References

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475.

Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*.

Chen, M.; Wang, J.; Liu, Z.; Wang, Y.; Zheng, H.; and Wang, Q. 2026. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324.

Kim, M.; Wang, D.; Cui, S.; Farmahinifarahani, F.; Zhuo, T. Y.; Garg, S.; Ray, B.; Mukherjee, R.; and Kumar, V. 2026. Coherence Collapse: Diagnosing Why Code Agents Fail After Reaching the Right Code. arXiv:2603.24631.

Mehta, A.; and Datta, A. 2026. Plans Don't Persist: Why Context Management Is Load Bearing for LLM Agents. arXiv:2606.22953.

Mehtiyev, T.; and Assunção, W. 2026. Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure. arXiv:2604.02547.

Oderinwale, H. 2026. Agent Trajectories as Programs: Fingerprinting and Programming Coding-Agent Behavior. arXiv:2606.16988.

Ou, T.; Guo, W.; Gandhi, A.; Neubig, G.; and Yue, X. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 207–215.

Wang, M.; Xie, X.; and Huo, Y. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563.

Zhou, C.; Lu, X.; Zhao, J.; Lin, J.; Ge, D.; and Ye, Y. 2026. ORSpace: A Full-Lifecycle Workspace Benchmark for Industrial Optimization Agents. arXiv:2605.28158.

Zhu, J.; Ro, Y.; Robertson, J. T.; Wang, K.; Li, J.; Vikalo, H.; Akella, A.; and Wang, Z. 2026. Your Agents Are Aging Too: Agent Lifespan Engineering for Deployed Systems. arXiv:2605.26302.